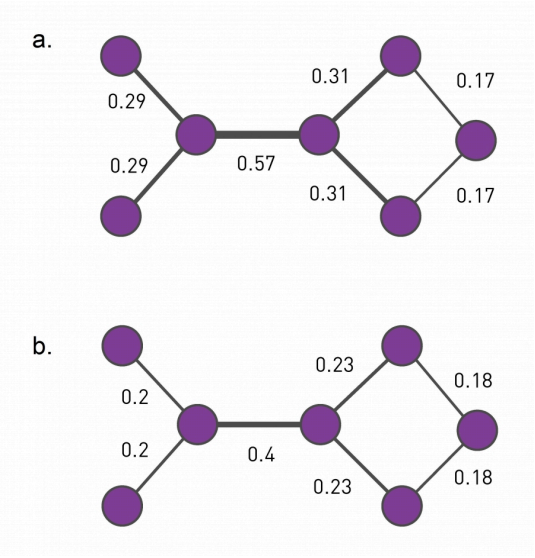
L7: Community Detection Based on Edge Centrality Metrics



Three centrality measures that are discussed in this image (Image 9.11) from networksciencebook.com.

A family of algorithms for community detection is based on hierarchical clustering. The goal of such approaches is to create a tree structure, or "dendrogram", that shows how the nodes of the network can iteratively split in a top-down manner into smaller and smaller communities ***("divisive hierarchical clustering")*** – or how the nodes of the network can be iteratively merged in a bottom-up manner into larger and larger communities ***("agglomerative hierarchical clustering").***

Let us start with divisive algorithms. We first need a metric to select edges that fall between communities – the iterative removal of such edges will gradually result in smaller and smaller communities.

One such metric is the edge *(shortest path)* betweenness centrality, introduced in Lesson-6. Recall that this metric is the fraction of shortest paths, across all pairs of terminal nodes, that traverses a given edge. Visualization **(a)** shows the value of the betweenness centrality for each edge. Removing the edge with the maximum centrality value *(0.57)* will partition the network into two communities. We can then recompute the betweenness centrality for each remaining edge, and repeat the process to identify the next smaller communities. This algorithm is referred to as **"Girvan-Newman"** in the literature.

Another edge centrality metric that can be used for the same purpose is the random walk betweenness centrality. Here, instead of following shortest paths from a node *u* to a node *v*, we compute the probability that a random walk that starts from *u* and terminates at *v* traverses each edge *e*, as shown in visualization (b). The edge with the highest such centrality is removed first.

Note that the computational complexity of such algorithms depends on the algorithm we use for the computation of the centrality metric. For betweenness centrality, that computation can be performed in $O(LN)$, where L is the number of edges and N is the number of nodes. Given that we remove one edge each time, and need to recompute the centrality of the remaining edges in each iteration, the computational complexity of the Girvan-Newman algorithm is $O(L^2 N)$. In sparse networks the number of edges is in the same order with the number of nodes, and so the Girvan-Newman algorithm runs in $O(N^3)$.